

Computational Work 1: Matrix Norms and SVD

Éderson Passos Silva

Course: Computational Matrix Methods

Professor: Cibele Aparecida Ladeia

Post-Graduate Program in Applied Mathematics (PPGMAP)

Abstract

This computational work investigates matrix norms, singular value decomposition (SVD), matrix reconstruction, and low-rank approximation through two numerical experiments. First, for a pseudo-random matrix $A \in \mathbb{R}^{8 \times 16}$ with entries in $-1, 0, 1$, the induced 1-, 2-, and ∞ -norms and the Frobenius norm are computed. The relationship between the induced 2-norm and the largest singular value is then established, followed by the full SVD, reconstruction error analysis, and a rank-4 approximation. Second, SVD is applied to a 200×200 matrix representing a structured mathematical image. The singular-value spectrum and Frobenius reconstruction errors are used to examine the effect of rank truncation. The numerical results illustrate the connection between singular values, matrix norms, exact reconstruction, and low-rank representations, with the computations performed in Python using NumPy.

1 Problem

The computational work proposed in the course is divided into two parts. The first part considers a pseudo-random matrix $A \in \mathbb{R}^{8 \times 16}$, whose entries are generated from the set $\{-1, 0, 1\}$. The required tasks are to compute the induced 1-, 2-, and ∞ -norms and the Frobenius norm; establish the relationship between the singular values of A and $2|A|$; determine the full singular value decomposition:

$$A = U\Sigma V^T;$$

evaluate the reconstruction error after forming $A_{\text{recon}} = U\Sigma V^T$; and analyze the spectral and Frobenius norms of the error associated with a rank- k approximation $A_k = U_k \Sigma_k V_k^T$. All numerical results are to be reported with six significant figures.

The second part considers a matrix $A \in \mathbb{R}^{200 \times 200}$ interpreted as a grayscale mathematical image. The assignment specifies the construction of the image from two spatial components and requires the use of SVD to obtain the singular-value spectrum, analyze its relation to the structure of the image, and construct low-rank approximations for selected values of k . The Frobenius reconstruction error must be computed and plotted as a function of k , and the value of k producing a visually satisfactory approximation must be identified.

In the computational results adopted in this report, the second experiment uses the generated matrix:

$$A(X, Y) = \sin(aX) + \cos(bY),$$

as documented in Section 1. This formulation is the basis of the numerical values and figures presented in Question 2.

2 Methodology

The computations were performed in Python using NumPy for matrix generation, norm evaluation, singular value decomposition, reconstruction, and low-rank approximation. The complete computational implementation is available in the [GitHub repository associated with this work](#), which provides the scripts used to generate the matrices, perform the numerical experiments, and reproduce the reported results. The pseudo-random matrix in Question 1 was generated with the registration number used as the random seed:

$$A = \text{choice}(\{-1, 0, 1\}, 8 \times 16).$$

The seed was fixed so that the numerical experiment is reproducible.

For the first experiment, the induced matrix norms were evaluated from their standard definitions. In particular, the 1-norm was computed as the maximum absolute column sum, the ∞ -norm as the maximum absolute row sum, and the 2-norm through the largest singular value. The Frobenius norm was computed from the square root of the sum of the squared entries. The results obtained directly from these definitions were compared with the corresponding values returned by `numpy.linalg.norm`.

The SVD was computed with the full-matrix option,

$$A = U\Sigma V^T,$$

where $U \in \mathbb{R}^{8 \times 8}$ and $V \in \mathbb{R}^{16 \times 16}$ are orthogonal and $\Sigma \in \mathbb{R}^{8 \times 16}$ contains the singular values. The reconstructed matrix was formed as $A_{\text{recon}} = U\Sigma V^T$, and its discrepancy from the original matrix was measured using both the Frobenius and spectral norms. A rank-4 approximation was subsequently obtained from the four largest singular values and corresponding singular vectors. The spectral and Frobenius norms of the resulting residual were then compared with the theoretical properties of the truncated SVD.

For Question 2, a 200×200 matrix was generated on a regular grid over $[0, 2\pi] \times [0, 2\pi]$. In the implementation corresponding to the results reported here, the matrix is

$$A(X, Y) = \sin(aX) + \cos(bY),$$

with the parameters generated from the fixed registration seed. Its SVD was computed and the singular values were examined as a function of their index. Rank- k approximations

$$A_k = U_k \Sigma_k V_k^T$$

were constructed for $k \in \{1, 2, 5, 10, 20, 50, 100\}$. For each selected rank, the Frobenius norm of the residual $A - A_k$ was calculated. The corresponding images were also compared visually to identify the smallest rank that provides a satisfactory reconstruction.

All reported numerical values are given with six significant figures, as required by the assignment. The computational implementation used double-precision floating-point arithmetic through NumPy.

3 Results Discussion

3.1 Question 1 – Analytical and Computational Evaluation of Matrix Norms

Item (a) – Computation of Induced and Entrywise Matrix Norms

We compute the induced p -norms ($p = 1, 2, \infty$) and the Frobenius norm for a pseudo-randomly generated matrix $A \in \mathbb{R}^{8 \times 16}$, whose entries are sampled from the discrete set $\{-1, 0, 1\}$.

The induced norms are defined by the action of A on vectors, whereas the Frobenius norm is an entrywise norm. The formulas used are

1. **Induced 1-Norm:**

$$\|A\|_1 = \max_{1 \leq j \leq 16} \sum_{i=1}^8 |a_{ij}|. \quad (1)$$

2. **Induced ∞ -Norm:**

$$\|A\|_\infty = \max_{1 \leq i \leq 8} \sum_{j=1}^{16} |a_{ij}|. \quad (2)$$

3. **Induced 2-Norm:**

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sigma_{\max}(A). \quad (3)$$

4. **Frobenius Norm:**

$$\|A\|_F = \sqrt{\sum_{i=1}^8 \sum_{j=1}^{16} |a_{ij}|^2} = \sqrt{\text{tr}(A^T A)}. \quad (4)$$

The values were computed using the `numpy.linalg` routines. All numerical results are reported with six significant figures.

The Python script used for the computation is provided in Appendix A.

Table 1: Numerical Results for the Norms of Matrix A

Matrix Norm Type	Computed Value (6 Significant Figures)
$\ A\ _1$	8.00000
$\ A\ _2$	4.50064
$\ A\ _\infty$	13.0000
$\ A\ _F$	9.32738

Note: The numerical values in Table 1 correspond to the random generation seed 'registration'. The values depend on the selected seed, while the definitions and norm properties remain unchanged.

Item (b) – Relationship Between Singular Values and the Matrix 2-Norm

There is a direct relationship between the singular values of $A \in \mathbb{R}^{m \times n}$ and its induced 2-norm. According to [1, Sections 2.3 and 2.4], the induced 2-norm is

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \max_{\|x\|_2=1} \|Ax\|_2. \quad (5)$$

For any vector x ,

$$\|Ax\|_2^2 = (Ax)^T (Ax) = x^T (A^T A) x. \quad (6)$$

Let the SVD of A be $A = U \Sigma V^T$. Then

$$A^T A = V (\Sigma^T \Sigma) V^T, \quad (7)$$

where the eigenvalues of $A^T A$ are the squared singular values of A :

$$\lambda_i(A^T A) = \sigma_i^2. \quad (8)$$

Therefore,

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{\sigma_{\max}^2} = \sigma_{\max}(A). \quad (9)$$

Thus, the induced matrix 2-norm is exactly the largest singular value. For the present matrix, this gives $\|A\|_2 = 4.50064$.

Item (c) – Full Singular Value Decomposition ($A = U \Sigma V^T$)

The full SVD factors $A \in \mathbb{R}^{8 \times 16}$ as $A = U \Sigma V^T$. The dimensions and properties of the factors are

- $U \in \mathbb{R}^{8 \times 8}$ is orthogonal and its columns are left singular vectors.
- $\Sigma \in \mathbb{R}^{8 \times 16}$ is diagonal in the rectangular sense and contains the singular values $\sigma_1 \geq \dots \geq \sigma_8 \geq 0$.
- $V \in \mathbb{R}^{16 \times 16}$ is orthogonal and its columns are right singular vectors.

The decomposition was computed with `numpy.linalg.svd` using `full_matrices=True`. The routine returns V^T as its third output, so V was obtained by transposition.

The matrices are reported below with six significant figures.

Matrix $U \in \mathbb{R}^{8 \times 8}$:

$$\begin{bmatrix} -0.040564 & 0.432060 & 0.713157 & -0.236277 & 0.187251 & 0.228345 & -0.219299 & -0.334608 \\ 0.579674 & -0.390330 & 0.316348 & -0.024141 & 0.063849 & -0.219010 & 0.502570 & -0.326104 \\ 0.519370 & -0.066446 & 0.014082 & 0.297512 & -0.249028 & 0.714517 & -0.203206 & 0.152595 \\ -0.048485 & 0.401202 & -0.214864 & 0.689778 & 0.292818 & 0.051300 & 0.264421 & -0.395517 \\ -0.542097 & -0.282409 & 0.322186 & 0.123371 & -0.008113 & 0.411416 & 0.535808 & 0.225683 \\ 0.005531 & -0.176876 & -0.436385 & -0.468922 & 0.535410 & 0.444239 & 0.061625 & -0.265621 \\ -0.093524 & 0.245208 & -0.218985 & -0.285947 & -0.712752 & 0.141745 & 0.294453 & -0.431968 \\ 0.295956 & 0.569581 & -0.052683 & -0.249819 & 0.135493 & 0.004839 & 0.458836 & 0.542117 \end{bmatrix}$$

Matrix $\Sigma \in \mathbb{R}^{8 \times 16}$:

$$\begin{bmatrix} 4.500640 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 4.105084 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 4.004045 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 0.000000 & 3.633095 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 0.000000 & 0.000000 & 3.286685 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 2.622460 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 1.486326 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \\ 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.878642 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 & 0.000000 \end{bmatrix}$$

Matrix $V \in \mathbb{R}^{16 \times 16}$:

$$\begin{bmatrix} 0.184315 & 0.192003 & 0.352945 & -0.011372 & -0.045279 & -0.163993 & 0.457828 & 0.167333 & -0.229450 & -0.147883 & 0.558572 & 0.116964 & -0.111113 & 0.014706 & -0.260244 & -0.239634 & 0.072037 & 0.180644 & -0.206126 & -0.313679 & 0.282365 & -0.258100 & 0.126391 & -0.115841 & -0.361742 & 0.420429 & -0.105173 & 0.357203 & 0.027750 & 0.326329 & 0.288264 & -0.094485 \\ -0.151338 & 0.147300 & -0.365469 & 0.182833 & -0.204167 & 0.070052 & 0.185426 & -0.189808 & -0.328394 & -0.269774 & -0.065650 & -0.367348 & 0.422717 & 0.277468 & -0.053856 & -0.304875 & 0.238011 & -0.021203 & 0.106499 & -0.180098 & -0.446960 & -0.375304 & -0.043618 & -0.367176 & 0.219040 & -0.200164 & 0.022606 & 0.312789 & 0.447884 & -0.080963 & 0.089871 & 0.141959 \\ -0.383666 & 0.109726 & 0.175021 & -0.374887 & -0.195106 & 0.089496 & 0.031742 & 0.032018 & 0.110869 & 0.328981 & 0.032773 & -0.115506 & 0.160705 & 0.355357 & -0.486190 & 0.313141 & -0.070573 & -0.410068 & -0.190985 & -0.147992 & -0.319484 & 0.544311 & 0.124281 & -0.149438 & -0.177043 & 0.120420 & 0.073518 & 0.433028 & -0.076454 & -0.173473 & -0.110879 & -0.213000 \\ -0.300943 & -0.132729 & -0.247476 & 0.058552 & -0.041855 & -0.277866 & -0.362572 & -0.038695 & 0.330839 & -0.098729 & 0.368624 & 0.160602 & -0.252462 & 0.370382 & -0.038630 & -0.363231 & 0.026065 & -0.161089 & -0.317415 & -0.369741 & 0.095075 & -0.366579 & -0.212671 & 0.098136 & -0.212189 & 0.098003 & 0.097527 & -0.258532 & 0.090023 & -0.516906 & -0.340767 & -0.123421 \\ 0.440647 & -0.052063 & -0.298194 & -0.091510 & 0.093281 & 0.116236 & 0.338630 & 0.241181 & 0.592823 & 0.198376 & -0.035988 & -0.094707 & 0.195010 & 0.109213 & -0.087543 & -0.228250 & 0.131013 & 0.592246 & 0.009629 & 0.161003 & -0.285200 & 0.192224 & -0.339628 & -0.115338 & 0.048070 & 0.455371 & 0.199664 & -0.072103 & 0.003406 & -0.226030 & 0.074546 & -0.218472 \\ 0.208232 & -0.415004 & -0.027737 & 0.287748 & 0.062321 & 0.045979 & -0.157857 & 0.057989 & -0.189510 & 0.284177 & 0.520579 & -0.137891 & 0.315255 & 0.183439 & 0.150291 & 0.322884 & -0.131787 & 0.130654 & -0.201792 & 0.390609 & -0.150210 & -0.153396 & -0.054144 & 0.604132 & -0.089627 & 0.029424 & -0.179311 & 0.458569 & 0.230183 & -0.064900 & -0.192992 & 0.111746 \\ -0.362423 & 0.142639 & 0.068093 & 0.033175 & 0.579702 & 0.189916 & 0.032791 & -0.187320 & 0.191599 & -0.080054 & 0.226000 & 0.230846 & 0.462744 & -0.258481 & -0.023651 & -0.072739 & 0.304596 & 0.155789 & 0.032679 & -0.404794 & 0.098708 & 0.354333 & -0.472489 & 0.300824 & -0.139803 & -0.407472 & 0.053968 & 0.090951 & 0.139908 & 0.227901 & 0.003461 & -0.026475 \\ -0.374653 & 0.004476 & -0.003088 & -0.309853 & -0.252079 & 0.002423 & 0.179286 & 0.412843 & 0.075161 & -0.017757 & 0.191991 & -0.155336 & 0.088364 & -0.155630 & 0.630850 & -0.022578 & 0.036098 & 0.311758 & -0.568078 & -0.048957 & 0.002429 & 0.084413 & 0.175100 & -0.131978 & 0.054387 & -0.212955 & 0.291678 & 0.057431 & -0.268621 & -0.071771 & -0.017702 & 0.553968 \end{bmatrix}$$

Item (d) – Matrix Reconstruction and Norm Comparison (A_{recon})

Using the computed factors, the reconstructed matrix is

$$A_{\text{recon}} = U\Sigma V^T. \quad (10)$$

In exact arithmetic, $A_{\text{recon}} = A$. In floating-point arithmetic, the computed factors and matrix products introduce roundoff errors. For IEEE 754 double precision, the unit roundoff is approximately $2^{-53} \approx 1.11 \times 10^{-16}$ [1, Section 2.7].

The residual $E = A - A_{\text{recon}}$ was evaluated using the Frobenius and spectral norms.

Table 2: Residual Error Norms Due to Floating-Point Arithmetic

Error Norm	Computed Value (6 Significant Figures)
$\ A - A_{\text{recon}}\ _F$	9.98568×10^{-15}
$\ A - A_{\text{recon}}\ _2$	6.19227×10^{-15}

As expected, $\|E\|_2 \leq \|E\|_F$. More generally, for a matrix of rank r ,

$$\|E\|_2 \leq \|E\|_F \leq \sqrt{r}\|E\|_2 [1, \text{Section 2.7}]. \quad (11)$$

For $r \leq 8$,

$$\frac{\|E\|_F}{\|E\|_2} = \frac{9.98568 \times 10^{-15}}{6.19227 \times 10^{-15}} \approx 1.612605, \quad (12)$$

which lies within the theoretical interval $[1, \sqrt{8}]$.

Item (e) – Low-Rank Approximation and Error Norm Analysis

A rank- k approximation obtained from the truncated SVD is defined by

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T. \quad (13)$$

Since the assignment does not prescribe a value of k , we selected $k = 4$, corresponding to half of the maximum possible rank of A . The Eckart–Young–Mirsky theorem states that the truncated SVD provides the best rank- k approximation in both the spectral and Frobenius norms [1, Theorem 2.4.8].

In particular,

$$\min_{\text{rank}(B)=k} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1}, \quad (14)$$

and

$$\min_{\text{rank}(B)=k} \|A - B\|_F = \|A - A_k\|_F. \quad (15)$$

Table 3: Residual Error Norms for the Rank-4 Approximation

Error Norm	Computed Value ($k = 4$, 6 Significant Figures)
$\ A - A_4\ _2$	3.28669
$\ A - A_4\ _F$	4.54541

The residual $E_k = A - A_k$ contains the singular values $\sigma_{k+1}, \dots, \sigma_p$, where $p = \min(m, n) = 8$. Consequently,

$$\|A - A_k\|_2 = \sigma_{k+1}, \quad (16)$$

while

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^p \sigma_i^2}. \quad (17)$$

Thus, the spectral norm is determined by the largest discarded singular value, whereas the Frobenius norm accounts for all discarded singular values [1, Theorem 2.4.8]. For $k = 4$, the computed

spectral error agrees with $\sigma_5 = 3.286685$, while the Frobenius error includes the contributions from $\sigma_5, \dots, \sigma_8$.

3.2 Question 2 – Matrix Patterns and SVD Image Compression

Item (a) – Singular Value Spectrum and Image Complexity

Using the registration seed, a matrix $A \in \mathbb{R}^{200 \times 200}$ was generated according to

$$A = \sin(aX) + \cos(bY), \quad (18)$$

where X and Y denote the corresponding grid coordinates.

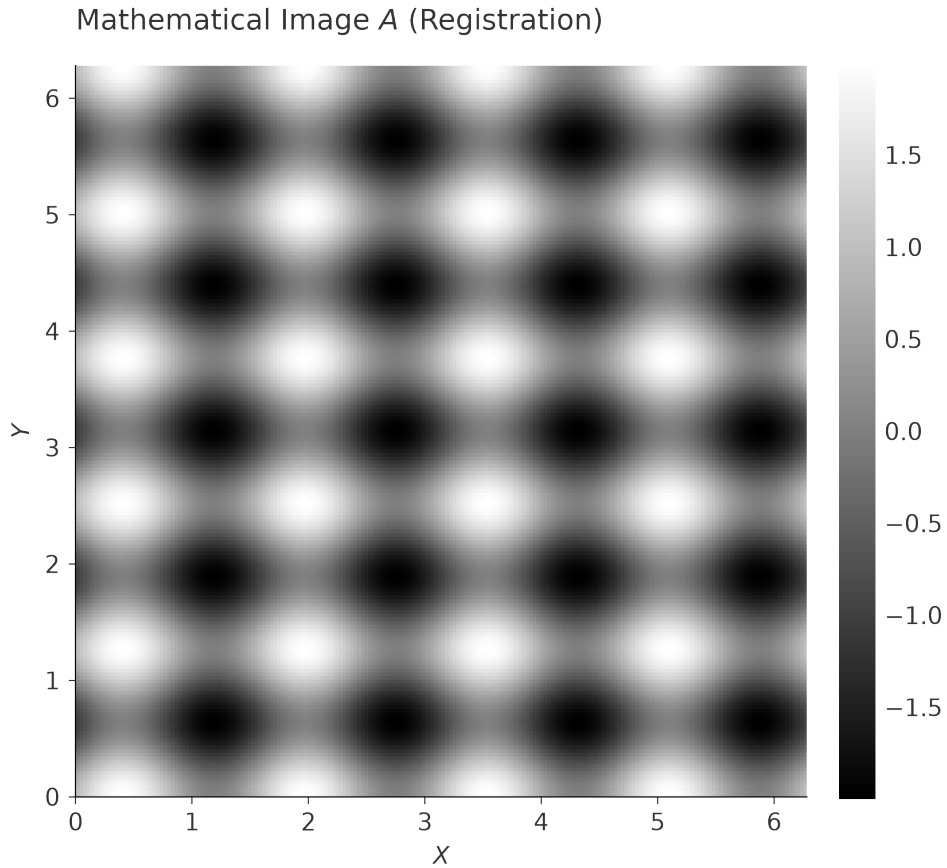


Figure 1: Visual Representation of the Input Matrix A. The matrix represents the function $A(X, Y) = \sin(aX) + \cos(bY)$ sampled on a regular 200×200 grid over $[0, 2\pi]$.

The singular value decomposition was then used to examine the rank structure of the matrix.

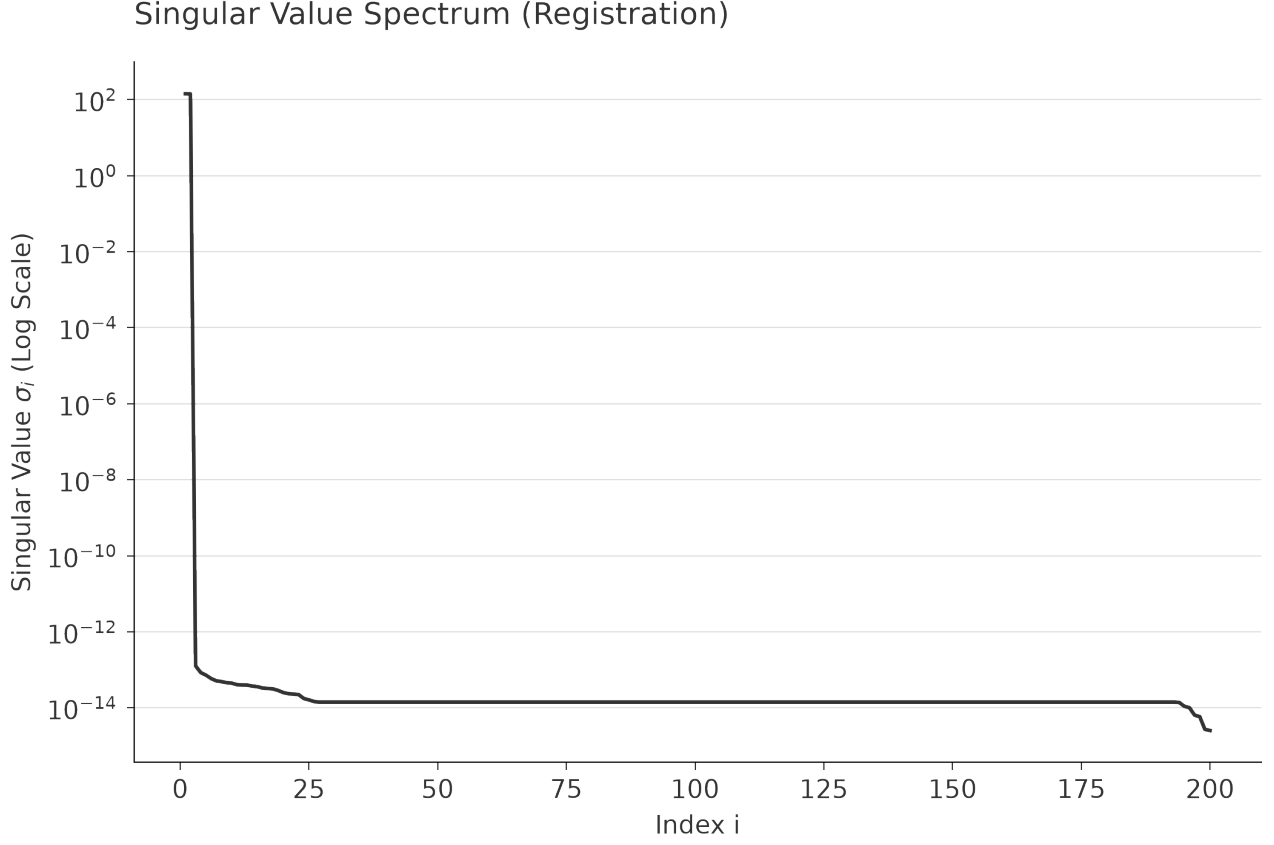


Figure 2: Singular Value Spectrum (σ_i) of the generated image matrix in logarithmic scale.

The matrix has a simple rank structure. The term $\sin(aX)$ depends only on the column coordinate and therefore generates a rank-1 matrix, while $\cos(bY)$ depends only on the row coordinate and also generates a rank-1 matrix. Hence,

$$\text{rank}(A) \leq \text{rank}(\sin(aX)) + \text{rank}(\cos(bY)) = 2. \quad (19)$$

The computed singular values confirm this structure. The two dominant values are

$$\sigma_1 = 142.031, \quad \sigma_2 = 140.809, \quad (20)$$

whereas

$$\sigma_3 = 1.26149 \times 10^{-13}. \quad (21)$$

The remaining singular values are likewise at the scale expected from numerical roundoff. Thus, the computed matrix is numerically rank 2, in agreement with its construction.

Items (b) and (c) – Low-Rank Approximation and Visual Satisfaction Threshold

Rank- k approximations

$$A_k = U_k \Sigma_k V_k^T \quad (22)$$

were computed for $k \in \{1, 2, 5, 10, 20, 50, 100\}$. The truncated SVD gives the best rank- k approximation in both the spectral and Frobenius norms [2, Lecture 5] and [1, Section 2.4]. The approximation error was measured using $\|A - A_k\|_F$.

Table 4: Frobenius Error Norm vs. Approximation Rank k

Rank k	$\ A - A_k\ _F$
1	140.809
2	1.10349×10^{-13}
5	2.01349×10^{-13}
10	2.30959×10^{-13}
20	2.56218×10^{-13}
50	2.70459×10^{-13}
100	2.89159×10^{-13}

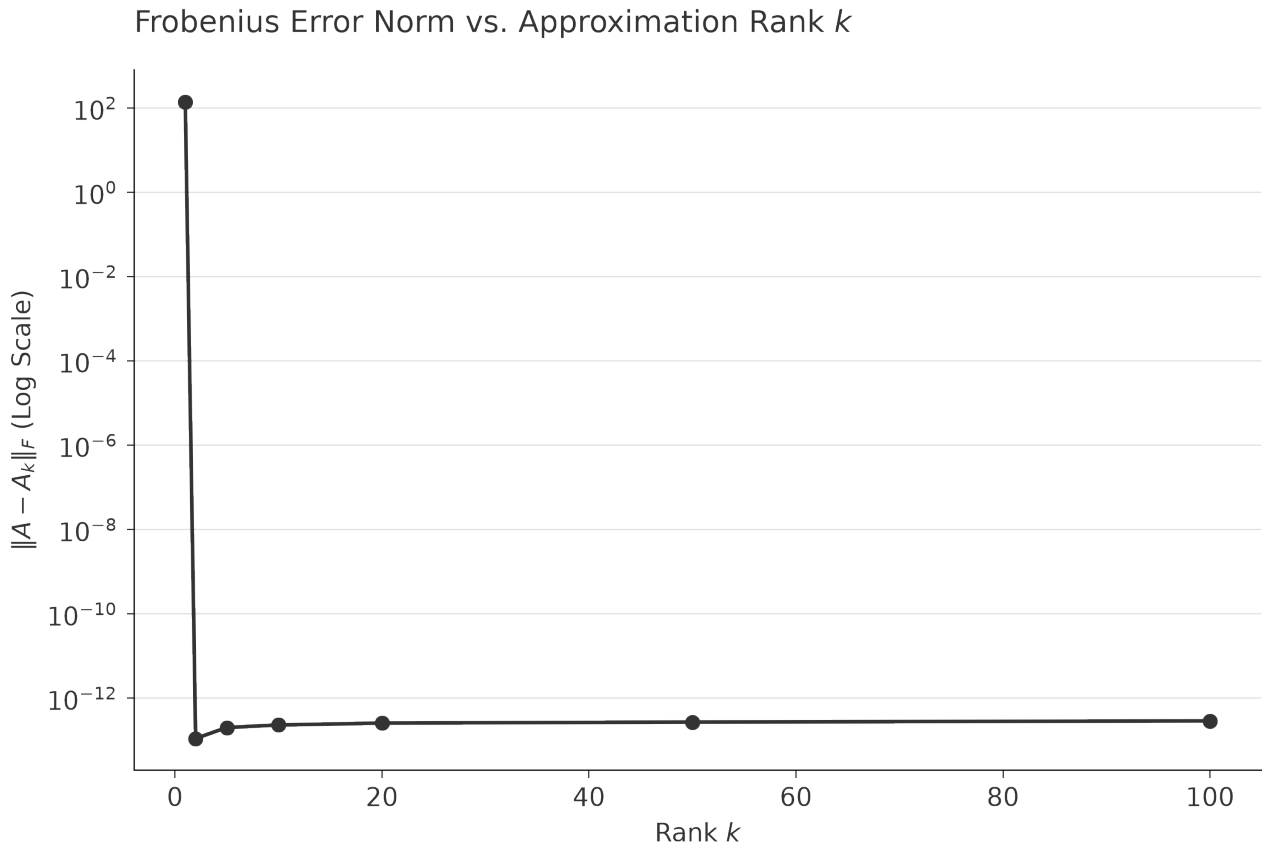


Figure 3: Decay of the Frobenius error norm $\|A - A_k\|_F$ as a function of the truncation rank k , plotted on a logarithmic scale.

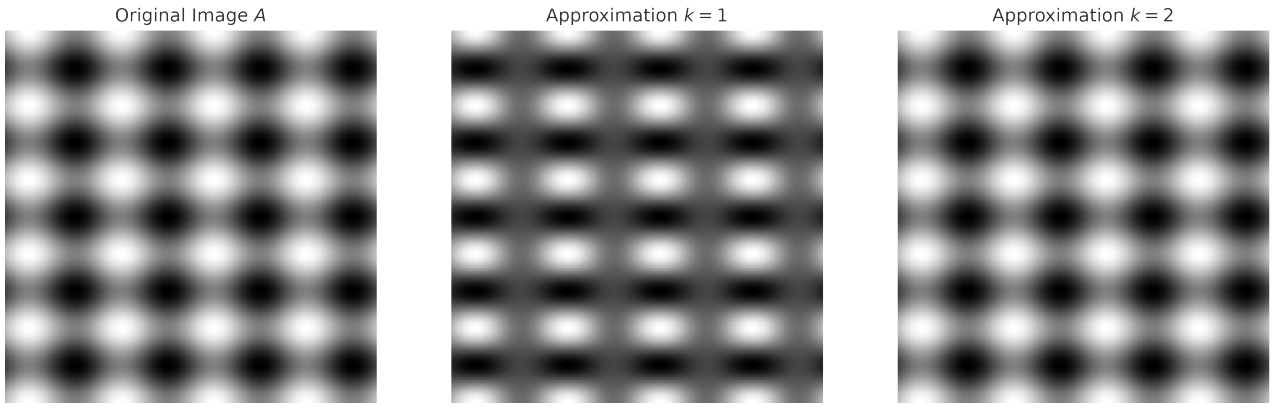


Figure 4: Visual comparison between the original matrix A and its low-rank approximations for $k = 1$ and $k = 2$.

The results follow directly from the rank structure established in Item (a). Each component of A is rank 1, so $\text{rank}(A) \leq 2$. Therefore, a rank-2 representation is sufficient to reproduce the matrix up to floating-point errors.

For $k = 1$, only one of the two dominant singular components is retained. Since σ_1 and σ_2 have similar magnitudes, the resulting approximation omits a substantial component of the matrix, producing the visible difference shown in Figure 4. This is consistent with the error $\|A - A_1\|_F = 140.809$.

For $k = 2$, the two dominant components are retained and the error decreases to approximately 10^{-13} . This value is consistent with the numerical precision of the computation rather than with a significant structural component omitted from the approximation.

For $k \geq 2$, the additional singular values correspond to components whose magnitudes are at the level of numerical roundoff. Consequently, retaining them does not provide a meaningful improvement in the approximation of the analytically rank-2 matrix. The small variations in the reported errors for larger k should therefore be interpreted as numerical effects associated with floating-point computation rather than as additional structure in the matrix.

References

- [1] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013. Consulted: Section 2.3 (Matrix Norms), Section 2.4 (The Singular Value Decomposition), and Section 2.7 (Finite Precision Matrix Computations).
- [2] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, 1997. Consulted: Lecture 5 (More on the SVD).

A Python code from Question 1-a

```
1 import numpy as np
2
3 registration = 318375
4
5 # Seed with my number registration
6 np.random.seed(registration)
7
8 # 1. Generation of random matrix A
9 A = np.random.choice([-1, 0, 1], size=(8, 16))
10
11 # 2. Manual calculation (via theoretical definitions) for numerical double-
    checking
12 norm_1_manual = np.max(np.sum(np.abs(A), axis=0))
13 norm_inf_manual = np.max(np.sum(np.abs(A), axis=1))
14 norm_F_manual = np.sqrt(np.sum(A**2))
15
16 # Calculation of norm-2 via SVD decomposition (square root of the largest
    eigenvalue of A^T A),
17 # where, ||A||_2 = max singular value
18 U, S, Vt = np.linalg.svd(A)
19 norm_2_manual = S[0]
20
21 # 3. Calculation via standard linear algebra library (Numpy via np.linalg.norm)
22 norm_1_lib = np.linalg.norm(A, ord=1)
23 norm_2_lib = np.linalg.norm(A, ord=2)
24 norm_inf_lib = np.linalg.norm(A, ord=np.inf)
25 norm_F_lib = np.linalg.norm(A, ord='fro')
26
27 # 4. Strict formatting with 6 significant figures
28 print(f"Matrix A Shape: {A.shape}")
29 print(f"||A||_1 = {norm_1_lib:.6g}") # Column Maximum Sum Standard
30 print(f"||A||_2 = {norm_2_lib:.6g}") # Maximum singular value of matrix A,
    via SVD decomposition
31 print(f"||A||_inf = {norm_inf_lib:.6g}") # Line Maximum Sum Standard
32 print(f"||A||_F = {norm_F_lib:.6g}") # Frobenius Norm
```